

Parallelizing Maximum Likelihood Estimation with Shared-Memory Threads

A C++ reformulation of Swann’s MPI multinomial-logit estimator

Youssef Nakhla Nicolas Asseo

CSE305 - Concurrent and Distributed Computing

June 2026

1 Introduction

Estimating econometric models has become more expensive as models grow more complex, and parallel computing is one natural way to deal with that. Our project starts from Swann’s paper, which estimates a multinomial logit model by maximum likelihood and runs it in parallel with MPI. In his design a master process splits the data, sends pieces to worker processes that each have their own private memory, who then send back their partial likelihoods and gradients to be combined.

This works because the log-likelihood is just a sum over independent observations. As a consequence, each observation’s contribution can be dealt with independently in parallel. Then, we only need to combine the partial results. Swann communicates these partials using messages, but we ask another question that fits the focus of this course. What does the same computation look like when the partial sums live in shared memory?

In practice, we replace MPI’s `MPI_SEND` and `MPI_RECV` with forks and joins. The pieces Swann sends to workers become ranges of indices into one shared array, and the step where the master adds everything up becomes a loop over the per-thread results.

We also build a second, Poisson-based model, which predicts counts (like doctor visits in a year), to check that the approach is not tied to logit. Both models have the same sum-over-observations shape, so the second one reuses our serial driver, both parallel backends and the optimizer with no changes. Only the likelihood and gradient differ.

The rest of the report is organized as follows. Section 2 sets out the model and the math we implement. Section 3 covers the serial baseline and the optimizer. Section 4 presents the two parallel versions and analyzes them with the work/depth framework. Section 5 deals with correctness, including why reordering floating-point sums matters. Section 6 reports and interprets the benchmarks for both models, and Section 7 looks at the Poisson application on its own. Section 8 concludes. Team responsibilities and build instructions follow at the end.

2 The Model and the Computation

We consider a multinomial logit model with $K = 5$ choices and $P = 5$ regressors (intercept, age, years of education, race, number of children) per observation, matching the structure of Swann’s application (a model of women’s annual decisions about marriage, work, and welfare participation). For observation i with regressor row $x_i \in \mathbb{R}^P$ and a parameter β_j (vector of five

weights indicating how each regressor influences choice j), the utility of choice j is $x_i\beta_j$ plus an i.i.d. extreme-value error. This yields the standard softmax choice probabilities

$$P_{ij}(\beta) = \frac{\exp(x_i\beta_j)}{\sum_{k=0}^{K-1} \exp(x_i\beta_k)}. \quad (1)$$

Due to the definition of softmax, $P(\beta) = P(\beta + c)$ with c a constant, so infinitely many β 's yield the same prediction. To fix this, Swann fixes $\beta_0 = 0$. That way, other choices are relative to this base choice. Therefore, there are $4 \cdot 5 = 20 = \text{NPARAM}$ free parameters. The log-likelihood over N observations is

$$\ln L(\beta) = \sum_{i=1}^N \ln P_{i,c_i}(\beta), \quad (2)$$

where c_i is the category chosen by observation i . Maximum likelihood estimation finds the β that maximizes the log-likelihood

$$\ln L(\beta) = \sum_{i=1}^N \ln P_{i,c_i}(\beta).$$

For the optimizer we also need the gradient. Differentiating the log-likelihood

$$\ln L(\beta) = \sum_{i=1}^N \ln P_{i,c_i}(\beta)$$

gives, for each choice $j \geq 1$ and regressor p ,

$$\frac{\partial \ln L}{\partial \beta_{j,p}} = \sum_{i=1}^N (\mathbf{1}[c_i = j] - P_{ij}(\beta)) x_{ip}. \quad (3)$$

Both the log-likelihood

$$\ln L(\beta) = \sum_{i=1}^N \ln P_{i,c_i}(\beta)$$

and the gradient

$$\frac{\partial \ln L}{\partial \beta_{j,p}} = \sum_{i=1}^N (\mathbf{1}[c_i = j] - P_{ij}(\beta)) x_{ip}$$

are sums over i , which is the property we exploit: every term depends only on observation i and the shared parameter vector β , so the terms can be computed in parallel with no interaction.

Numerical stability. We compute the softmax

$$P_{ij}(\beta) = \frac{\exp(x_i\beta_j)}{\sum_{k=0}^{K-1} \exp(x_i\beta_k)}$$

by subtracting the maximal linear index before exponentiating (i.e. the $x_i\beta_j$ which yields the highest score), which avoids overflow without changing the result. The same stabilization is used when generating data.

Synthetic data. We generate synthetic data mimicking the data used by Swann (mean age ≈ 26 , education ≈ 12 years, etc.). A “true” parameter vector β_{true} is drawn once from a uniform distribution and held fixed; it defines the softmax probabilities from which each observation’s choice is sampled. In a real application one would use actual survey data instead. The generator is seeded so every run sees exactly the same dataset, which is essential for comparing results.

3 Serial Baseline and Optimizer

The serial driver is a straight reading of the log-likelihood

$$\ln L(\beta) = \sum_{i=1}^N \ln P_{i,c_i}(\beta),$$

and the gradient

$$\frac{\partial \ln L}{\partial \beta_{j,p}} = \sum_{i=1}^N (\mathbf{1}[c_i = j] - P_{ij}(\beta)) x_{ip}.$$

It loops over every observation and adds up the log-likelihood and, when asked, the gradient. We kept it deliberately plain because it plays two roles. It is the reference we trust for correctness and it is the baseline every speedup is measured against.

To turn single evaluations into an actual estimate we use gradient ascent with a backtracking line search. The logit log-likelihood is globally concave, so there are no local maxima to get stuck in and the only real difficulty is picking a step size which is taken care of by the line search. At each step we move along the normalized gradient, start with a step of one and keep halving it until the likelihood actually goes up. We chose not to use Newton’s method, since that needs the Hessian as well as forming and inverting it which is a lot of linear algebra that has nothing to do with the concurrency question we care about.

Our first stopping rule looked at the gradient norm and stopped once it dropped below a fixed value but it never hit. The gradient

$$\frac{\partial \ln L}{\partial \beta_{j,p}} = \sum_{i=1}^N (\mathbf{1}[c_i = j] - P_{ij}(\beta)) x_{ip}$$

is a sum over all N observations, so it grows with the dataset and no single threshold works for every size. We switched to a rule that does not depend on scale and stop when the relative improvement in the log-likelihood from one step to the next falls below a tolerance. After that the optimizer converged in 19 to 31 iterations across every size we tried and the average log-likelihood per observation settled around -0.13 no matter how big N was which told us the rule behaves the same way at every scale.

The optimizer never calls a specific likelihood function. It takes an evaluator, a `std::function` that maps a parameter vector and a `with_grad` flag to a result and calls that instead. One optimizer runs on top of the serial, mutex or partial-sum backend with no change, ensuring that the timing comparisons are fair. We just wrap the backend we want in a lambda where we call it. Because the optimizer logic, the iteration count and the convergence path are all identical, any difference in solve time comes purely from the parallel strategy. The same trick lets the Poisson application in Section 7 reuse the optimizer untouched.

4 Parallel Implementations

Each optimization loop is done sequentially, but each iteration computes the likelihood and the gradient for the current β . As explained in Section 2, both of these are sums over N observations, so this is what we parallelize. This mirrors Swann’s structure, where the master updates parameters serially between parallel likelihood evaluations. In our implementation, we developed two versions of this parallelization: one using locks, and one using partial sums.

4.1 Version 1: shared accumulator with a mutex

Our first version keeps a single shared result and protects it with a mutex. Each thread processes a contiguous chunk of observations and, for every observation, locks the mutex to add its contribution to the shared totals:

```
1 void work(const Dataset& d, const Params& beta, int repeat, bool with_grad,
2         size_t num_threads, Result& res, std::mutex& m, size_t thread_id) {
3     // we define begin and end here in the original code
4     for (std::size_t i = begin; i < end; ++i) {
5         double ll = loglik_one(d, beta, i, repeat);
6         std::lock_guard<std::mutex> lock(m);
7         res.loglik += ll;
8         if (with_grad) grad_one(d, beta, i, repeat, res.grad);
9     }
10 }
```

Version 1: a lock is taken on every observation.

This works correctly: the mutex prevents the read-modify-write race on `res.grad` and `res.loglik` but it acquires the lock once per observation. However, `grad_one` runs inside the lock, which means the gradient update (which loops over all 20 parameters) is also serialized for each observation. So in the end, threads spend most of their time waiting for the lock rather than doing useful, computational work. We still include this because it is a great example of excessive synchronization, and its impact shown in Section 6 demonstrates this.

4.2 Version 2: per-thread partial sums

The optimized version eliminates the shared mutex. Instead, each thread owns a private slot in which it accumulates its own partial log-likelihood and gradient. After all threads have joined, the main thread combines each independent result:

```
1 std::vector<double> partial_ll(num_threads, 0.0);
2 std::vector<std::vector<double>> partial_grad(num_threads,
3         std::vector<double>(NPARAM, 0.0));
4
5 void work_v2(const Dataset& d, const Params& beta, int repeat, bool with_grad,
6         size_t num_threads, std::vector<double>& partial_ll,
7         std::vector<std::vector<double>>& partial_grad, int thread_id) {
8     // we define begin and end here in the original code
9     double local_ll = 0.0;
10    for (std::size_t i = begin; i < end; ++i) {
11        local_ll += loglik_one(d, beta, i, repeat);
12        if (with_grad) grad_one(d, beta, i, repeat, partial_grad[thread_id]);
13    }
14    partial_ll[thread_id] = local_ll; // single write to own slot
15 }
16 // ... launch threads, join all ...
17 for (int t = 0; t < num_threads; ++t) { // reduce
18     res.loglik += partial_ll[t];
19     for (int k = 0; k < NPARAM; ++k) res.grad[k] += partial_grad[t][k];
20 }
```

Version 2: each thread writes only its own slot. We reduce after joining.

There is no lock anywhere. Although the per-thread slots live in shared vectors, each thread writes only to the index matching its own id, so the threads touch disjoint memory locations and no race condition is possible. This is the shared-memory equivalent of Swann's strategy, who used messages to communicate partial results.

4.3 Work, depth, and expected speedup

Using lecture 2 which presented the PRAM model, the total cost of all the tasks is $T_1 = O(N)$, since each of the N observations costs a constant amount. For Version 2 with T threads, each thread sums its N/T observations sequentially, and the main thread then combines the T partial results, so the runtime is

$$T_T = O\left(\frac{N}{T} + T\right).$$

So the resulting speedup is

$$S = \frac{T_1}{T_T} = \frac{N}{\frac{N}{T} + T} = \frac{NT}{N + T^2},$$

Ideally, we would want $S = T$, which holds when $T \ll \sqrt{N}$. Within our work, this always holds since $T \leq 8$ and $N \geq 50,000$. This shows, in addition to our previous justification, that Version 2 is the most natural way to process this problem in parallel.

Version 1 has the same total work T_1 but performs much worse because the mutex which allows likelihood and gradient accumulation is acquired and locked for each observation, so

$$T_T = O\left(\frac{N}{T}\right) + O(N) = O(N).$$

So the resulting speedup is

$$S = \frac{T_1}{T_T} = \frac{N}{N} = 1,$$

This predicts exactly what we observe: Version 2 scales nearly linearly while Version 1 does not scale at all.

5 Correctness

We check correctness at three levels.

The first level is checking just the pure math. We wrote a small test program (`check_model.cpp`) that runs the functions computing one observation's log-likelihood and gradient and compares their output against values we worked out by hand. The easiest case is $\beta = 0$. There every choice is equally likely, so the log-likelihood has to equal $\ln(1/K)$ and the gradient has to equal the closed form

$$(\mathbf{1}[c_i = j] - 1/K) x_p.$$

The same program also checks the gradient against a finite-difference estimate at a β that is not zero. We compute the likelihood at β and at small steps on either side of it, take the slope and compare it to the gradient our code returns. The second check is also important because it catches sign slips and wrong indices that the $\beta = 0$ case would hide.

The second level is agreement between the serial and parallel code. A separate test program (`test_correctness.cpp`) runs both parallel versions against the serial baseline for 1, 2, 3, 4 and 8 threads and checks that the answers match within a small relative tolerance. Our benchmark output also has a `match` column that records this, and it came back positive for all configurations of each model in Section 6.

The third level deals with floating point. We compare with a relative tolerance rather than exact equality since addition of floating-point numbers is not associative, so summing each partial contribution in a different order (which happens when splitting across threads) can change the last few bits. This is not a bug, it is just what a parallel sum does and it is worth being clear that when we say the parallel answer equals the serial answer we mean up to rounding, not bit for bit.

6 Experimental Evaluation

We collected the numbers in this section on an Apple MacBook Pro with an 8-core CPU, built with -O2 in Release mode. We time the full solve, a complete gradient-ascent run that does dozens of likelihood and gradient evaluations. That is a heavier and more realistic workload than a single evaluation and it matches Swann’s setting where one estimation involves many parallel evaluations. To make sure each run lasts long enough to time well and is not overshadowed by the cost of creating threads, every observation’s contribution is computed with a fixed repetition factor. This inflates the work per observation without changing the result which is the same idea Swann uses in his Section 8 to get a realistic ratio of computation to communication. We report the best of three trials and keep each parallel version’s own best time. We sweep dataset sizes N in $\{50,000, 100,000, 200,000, 500,000\}$ and thread counts of 1, 2, 3, 4 and 8. The script can also rerun the sweep on other machines.

6.1 Logit model results

Table 1 gives the full-solve runtimes for all three logit versions together with how much faster each parallel version is than serial. The figures that follow show the same numbers.

Table 1: Logit full-solve runtimes (ms) and speedups over serial. “v1” is the mutex version, “v2” the partial-sum version. Best of three trials, 8-core MacBook Pro, -O2.

N	threads	serial	v1 (ms)	v2 (ms)	speedup v2	speedup v1
50,000	1	1249	1284	1238	1.01	0.97
50,000	2	1230	709	647	1.90	1.73
50,000	3	1229	692	448	2.74	1.77
50,000	4	1262	957	338	3.74	1.32
50,000	8	1262	2105	182	6.94	0.60
100,000	1	4312	4453	4326	1.00	0.97
100,000	2	4335	2475	2229	1.94	1.75
100,000	3	4344	2288	1513	2.87	1.90
100,000	4	4305	3291	1145	3.76	1.31
100,000	8	4297	7280	599	7.18	0.59
200,000	1	8525	8831	8561	1.00	0.97
200,000	2	8529	4779	4350	1.96	1.78
200,000	3	8528	4571	2989	2.85	1.87
200,000	4	8530	6282	2255	3.78	1.36
200,000	8	8533	14401	1172	7.28	0.59
500,000	1	23122	23889	23200	1.00	0.97
500,000	2	23107	12889	11761	1.96	1.79
500,000	3	23092	12288	8077	2.86	1.88
500,000	4	23085	17060	6087	3.79	1.35
500,000	8	23080	39371	3131	7.37	0.59

Version 2 scales well. It is about $1.96\times$ faster at 2 threads, $2.86\times$ at 3, $3.79\times$ at 4 and as much as $7.37\times$ at 8 threads on the largest dataset. Efficiency, which is speedup divided by thread count, stays above 90 percent through 4 threads and reaches 92 percent at 8 threads on the biggest problem (Figure 3).

Efficiency also gets better as the dataset grows. At 8 threads it climbs from 84 percent at $N = 50,000$ to 92 percent at $N = 500,000$. The reason is that creating the threads is a fixed cost paid once per evaluation, so the more work there is the smaller that cost looks. This is the

shared-memory version of Swann’s point that speedup improves when there is more computation per unit of communication, which we pin down with a separate experiment in Section 6.2.

Version 1 goes the other way. It manages about $1.8\times$ at 2 threads and even reaches around $1.9\times$ at 3, but then it falls apart. At 4 threads it is down to roughly $1.35\times$ and at 8 threads it sits around $0.59\times$, which means it is nearly twice as slow as plain serial code. With more threads fighting over a lock that is taken once per observation, the time lost to contention and to moving the shared totals between caches outweighs anything gained from running in parallel (Figure 4). This is also a clean demonstration of the course showing that synchronization is not free.

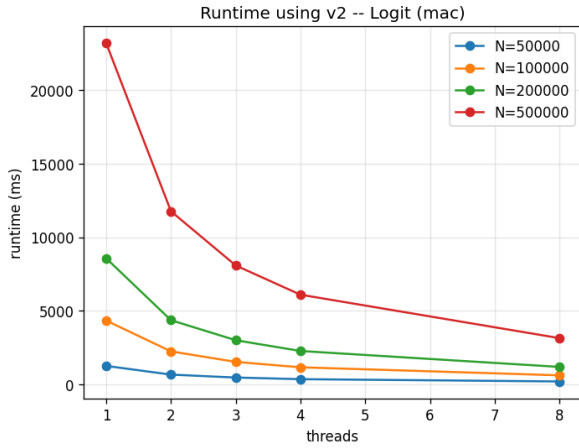


Figure 1: Logit: full-solve runtime (v2) versus thread count. Runtime falls monotonically as threads are added.

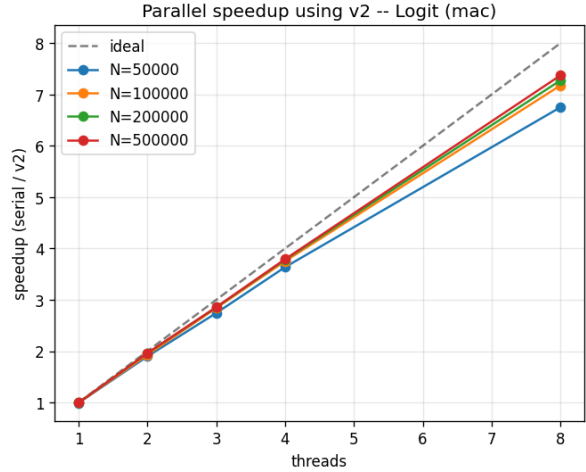


Figure 2: Logit: speedup of v2 over serial. All sizes track close to ideal and larger N is slightly better.

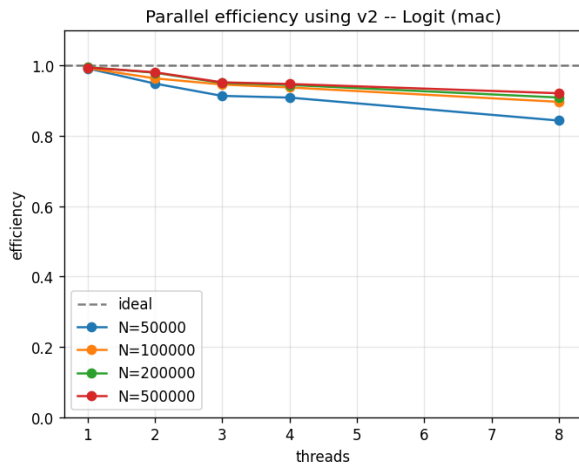


Figure 3: Logit: parallel efficiency (v2). Efficiency stays high and improves with N .

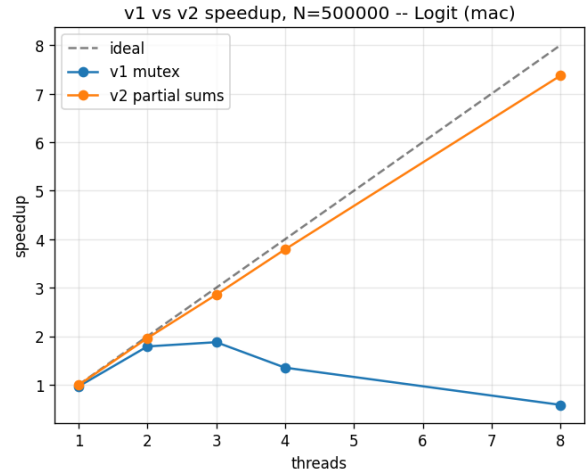


Figure 4: Logit: v2 scales toward ideal while v1 (mutex) collapses below $1\times$ past 3 threads.

6.2 Effect of the per-evaluation workload

To make the computation-versus-communication point concrete we fix $N = 200,000$ and 4 threads and vary the repetition factor that sets how much arithmetic each observation costs. Table 2 shows what happens. We did this for both the original logit model and the extra Poisson

model Section 7. As the work per evaluation rises the fixed fork/join overhead matters less and the speedup climbs steadily toward the 4-thread limit for both models (Figures 5 and 6). This is the direct shared-memory version of Swann’s Section 8 result that parallel gains grow as the compute rises against the communication.

Table 2: Speedup of v2 over serial as the per-observation workload (repeat factor) increases, at $N = 200,000$, $T = 4$.

repeat	speedup (logit)	speedup (Poisson)
1	3.60	3.07
10	3.72	3.25
50	3.79	3.63
100	3.80	3.66
200	3.82	3.75
500	3.83	3.79

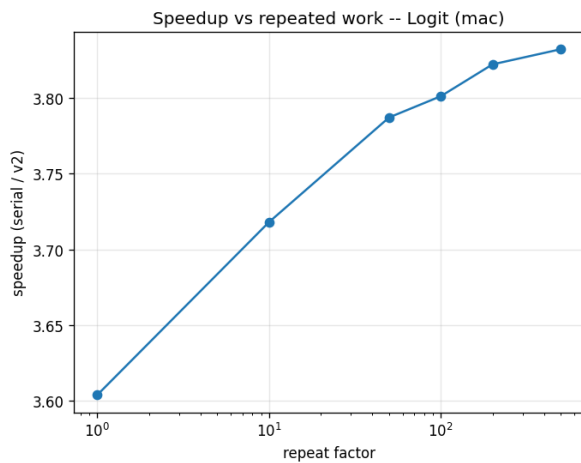


Figure 5: Logit: speedup vs. repeat factor (log x -axis), $N = 200,000$, $T = 4$.

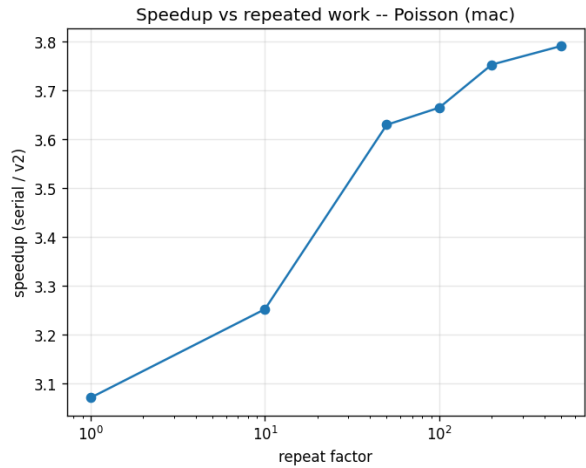


Figure 6: Poisson: speedup vs. repeat factor. The lighter Poisson kernel starts further from the ceiling but climbs more steeply.

We can observe a steeper curve on the poisson graph compared to the logit one. A single Poisson term is much cheaper than a logit term, since it is one dot product and one exponential rather than a full K -way softmax. So at repeat = 1 the overhead is a bigger share of the work and the speedup is only $3.07\times$. By repeat = 500 it has risen to $3.79\times$, almost the same as logit. The two models show the same effect.

With regards to correctness, the `match` column was positive for every configuration of both models. Every parallel solve reached the same log-likelihood as the serial solve (within rounding) so we can see that although performance changed a lot between versions, the estimates themselves did not.

We also designed the benchmarking script to detect which machine it is on and write results into a folder named after that machine, so the whole sweep can be rerun elsewhere. We ran it again on a Dell XPS15 under Windows Subsystem for Linux (WSL). The speedups and efficiency curves matched the Mac run: version 2 scaled well, efficiency improved with N , and the mutex version fell behind once a few threads were in use. We only keep the MacBook Pro tables and figures above as the main results because the WSL runs were much slower overall. This is due

to the fact that thread creation and join overhead showed up much more strongly there. This is shown by the fact that the recorded serial time varied a lot from one row to the next even though the serial code never uses more than one thread: at $N = 50,000$ the serial column ranged from about 27 to 43 seconds depending on T .

7 Second Application: Poisson Regression

To check that our design is not tied to one model we ran the same machinery on a Poisson regression. This is a standard model for counts, like the number of doctor visits a patient makes in a year. We generate covariates in the style of the RAND Health Insurance Experiment and draw each count as $y_i \sim \text{Poisson}(\mu_i)$ with $\mu_i = \exp(x_i\beta)$. The per-observation log-likelihood and gradient are

$$\ln L_i = y_i (x_i\beta) - e^{x_i\beta}, \quad \frac{\partial \ln L_i}{\partial \beta_p} = (y_i - e^{x_i\beta}) x_{ip}. \quad (4)$$

The important thing is the shape. This is again a sum over independent observations, so the serial driver, both parallel backends, the optimizer and the benchmark can be reused as they are. Only `loglik_one` and `grad_one` change, and the model has $\text{NPARAM} = P = 5$ parameters rather than 20.

Table 3 and the figures that follow give the Poisson results. The top-line numbers look like the logit case. Version 2 reaches $7.08\times$ at 8 threads for $N = 500,000$ while version 1 again drops below serial. But because each observation is so cheap, Poisson shows the limits of parallelism more clearly.

Table 3: Poisson full-solve runtimes (ms) and speedups over serial. Best of three trials, 8-core MacBook Pro, -O2.

N	threads	serial	v1 (ms)	v2 (ms)	speedup v2	speedup v1
50,000	1	94.9	129.9	97.1	0.98	0.73
50,000	2	93.1	238.7	50.8	1.83	0.39
50,000	3	93.2	826.5	37.7	2.47	0.11
50,000	4	94.3	365.6	29.9	3.12	0.26
50,000	8	93.9	313.3	20.4	4.56	0.30
100,000	1	186.1	259.2	188.1	0.99	0.72
100,000	2	186.1	467.2	98.7	1.88	0.40
100,000	3	185.6	1796	70.1	2.65	0.10
100,000	4	187.8	689.7	54.3	3.42	0.27
100,000	8	186.2	607.4	32.4	5.73	0.31
200,000	1	372.8	508.4	374.5	0.99	0.73
200,000	2	372.0	919.3	192.9	1.93	0.40
200,000	3	371.7	3337	134.3	2.77	0.11
200,000	4	371.5	1357	102.7	3.62	0.27
200,000	8	372.5	1178	57.7	6.44	0.32
500,000	1	929.5	1272	937.1	0.99	0.73
500,000	2	929.6	2274	478.2	1.94	0.41
500,000	3	929.5	9249	329.1	2.82	0.10
500,000	4	929.7	3748	248.8	3.73	0.25
500,000	8	929.3	2897	131.2	7.08	0.32

At $N = 50,000$ the Poisson version 2 hits only 57 percent efficiency at 8 threads, well under the 84 percent the logit model gets at the same point (compare Figures 9 and 3). That is

the computation-versus-communication trade-off again. A Poisson evaluation does much less arithmetic per observation than a logit one, so the fork/join overhead has a bigger share. As N grows the gap closes and efficiency rises to 88 percent at $N = 500,000$, because there is finally enough work per thread.

The mutex version is worse here than for logit. Its speedups are below $0.5\times$ everywhere and drop as low as $0.10\times$ at 3 threads, which is a tenfold slowdown. The useful work per observation is so small that the lock, taken once per observation with the gradient update held inside it, takes over almost completely.

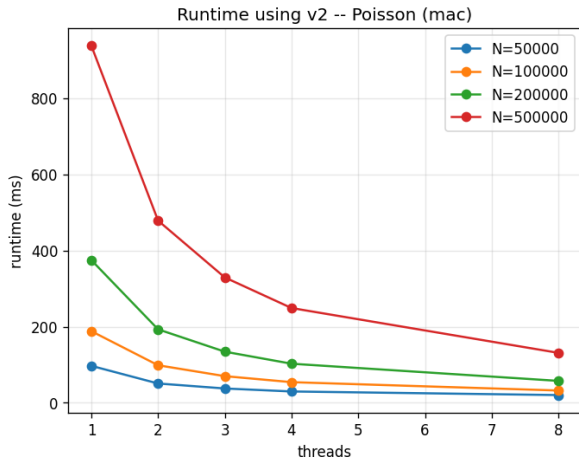


Figure 7: Poisson: full-solve runtime (v2) versus thread count.

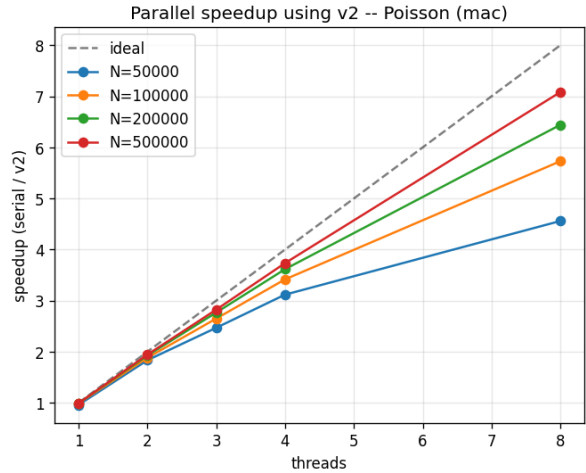


Figure 8: Poisson: speedup of v2 over serial. Lower at small N than logit, rising with N .

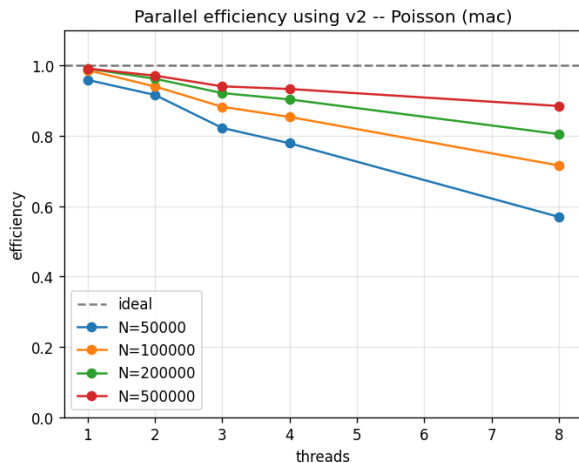


Figure 9: Poisson: parallel efficiency (v2). The small- N curves under perform because each observation is cheap.

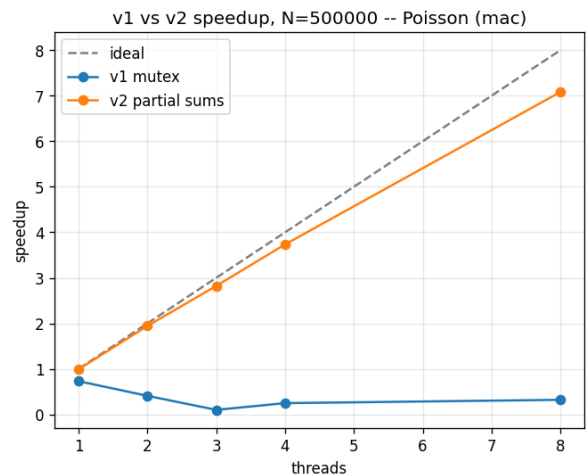


Figure 10: Poisson: the mutex version is much worse here, dipping to a $10\times$ slowdown.

8 Conclusion

We rebuilt Swann’s MPI estimator for the multinomial logit model as a shared-memory program. The message passing between a master and its workers was replaced by the fork/join pattern and the partial likelihoods and gradients that Swann sends as messages became per-thread slots that the main thread adds up after a join. The serial baseline, the optimizer and both parallel

versions sit behind one interface, so the only thing that changes between runs is how the work is divided.

The partial-sum version performs well. On the 8-core machine it reaches $7.37\times$ for the logit model and $7.08\times$ for the Poisson model at 8 threads and parallel efficiency stays above 88 percent once the dataset is large. In every variation it returns the same estimates as the serial baseline, up to floating-point rounding, so the speed comes at no cost to the result.

The mutex version behaves the opposite way. Once more than a couple of threads are running it is slower than the serial code and for the cheap Poisson kernel it reaches a tenfold slowdown. The two versions do the same arithmetic and differ only in how they combine results, so the whole difference comes from synchronization.

Reusing the same code for a second model showed how performance depends on the work per observation. The Poisson kernel does far less arithmetic per observation than the logit one so its fixed thread overhead is more prominent and its efficiency at small N is lower. We then used the repeat experiment to show this more strongly. As the work per evaluation grows, efficiency climbs back up. Speedup therefore depends not only on how many threads are used but on how much computation each observation carries.

Several extensions follow naturally. For example, Newton’s method would cut the number of iterations but would need much more heavy linear algebra for the Hessian computation. Also, a GPU implementation could also use the same per-observation decomposition across a much larger number of threads, with the main design challenge shifting to how the partial results are reduced efficiently.

Team Responsibilities

Youssef Nakhla worked mainly on the serial and mathematical core. He wrote the logit likelihood and gradient functions in `model.cpp` and built the serial driver in `serial.cpp`. He added the tests for the model math and the serial driver in `check_model.cpp` and fixed the `repeat` mechanism so it adds benchmark work without changing the result. He then wrote the reusable gradient-ascent optimizer with line search in `optimizer.cpp` along with its test in `check_optimizer.cpp` and added the solve-mode timing to the main program so a full optimization could be timed under each backend. He also built the benchmarking pipeline, including the plotting script and the final `run_all.sh` that runs the whole sweep for both models in one command and writes every CSV, plot and efficiency table to collect the final benchmark results.

Nicolas Asseo worked mainly on the scaffolding, the data generation and the parallel track. He set up the initial repository and project structure, and wrote the synthetic data generator in `data.cpp`. He wrote the first parallel version with a mutex in `parallel.cpp` and then replaced it with the per-thread partial-sum version in `parallel_v2.cpp`, and added the test comparing the parallel versions against the serial baseline in `test_correctness.cpp`. He also built the second application, a Poisson regression for count data, writing the Poisson likelihood and gradient in `applications/poisson/model.cpp` and the count-data generator in `applications/poisson/data.cpp` so that it reused the serial and parallel backends without change. He made later readability and bug-fix commits to `model.cpp` and reran the benchmark sweep on a Dell XPS15 under WSL to check that the Mac results carried over to another machine.

Both of us worked on debugging, reading the results and writing this report. Both of us committed regularly throughout the project.

Repository and Build Instructions

The code lives at <https://github.com/YoussefNakhla/Concurrent>. It builds with CMake and runs on the Salle Info machines.

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
./build/check_model          # math + serial unit tests
./build/check_optimizer      # optimizer convergence test
./build/test_correctness     # serial vs parallel agreement
./build/mle solve 100000 4 50 # logit full solve (N threads repeat)
./build/poisson_mle solve 100000 4 50 # poisson full solve
```

To regenerate every benchmark and plot for both models with one command, with results written to `results/<machine>/`:

```
./bench/run_all.sh
```

References

- [1] C. A. Swann, *Maximum Likelihood Estimation Using Parallel Computing: An Introduction to MPI*, Computational Economics **19** (2002), 145–178.